AI-Generated vs. Real Image Detection

Course: IEOR 142A Spring 2026 Project

Team: Theo Zhang

Interactive demo: https://huggingface.co/spaces/Theo9598/ai-vs-real-image-detector

GitHub appendix: https://github.com/Theo9598/ai-vs-real-image-detector

Primary dataset: Tiny GenImage 5k subset from https://huggingface.co/datasets/TheKernel01/Tiny-GenImage

Local comparison dataset: course project Google Drive dataset

Presentation recording: [Paste accessible recording link before Gradescope submission]

Abstract

This project studies AI-generated image detection as a supervised binary classification problem. The deployed model is a Random Forest trained on handcrafted image features from a 5,000-image Tiny GenImage subset. I compare it with Logistic Regression and a 10-epoch ResNet18 transfer-learning model, using validation-only threshold selection and a held-out 1,000-image test set. Random Forest achieves 98.2% test accuracy and 0.982 fake-class F1, outperforming Logistic Regression and ResNet18 on this subset. The main learning is that simple course models can be highly competitive when the feature representation fits the data, but the detector still has important generalization limits.

1 Problem and Data

The task is binary classification: label 0 means real and label 1 means AI-generated. The final demo uses Tiny GenImage because it is larger and more relevant to the final experiment than the small local folder dataset. I use 3,600 Tiny GenImage examples for fitting, 400 for validation, and 1,000 held-out examples for final testing. The Google Drive dataset with 995 images is kept as a smaller local comparison experiment, not as the active deployed model.

Table 1. Dataset protocols.

Dataset	Fit / Train	Validation	Test	Role in project
Tiny GenImage 5k	3,600	400	1,000	Primary experiment and deployed Random Forest model.
Local Google Drive dataset	695	150	150	Smaller comparison experiment and sanity check.

2 Models

Random Forest with handcrafted features.

Random Forest is the final deployed model. Each uploaded image is first converted into 63 handcrafted numerical features. The forest contains many decision trees; each tree votes using different feature splits, and the final probability is the average vote. This is useful here because the relationship between image artifacts and the fake label is nonlinear.

Table 2. Random Forest feature groups.

Feature group	What it measures	Why it may help
Aspect ratio and image size	Width/height ratio and log image area.	Some generated or collected images have repeated size/crop patterns.
RGB color means and standard deviations	Average and spread of red, green, and blue channels.	Synthetic images may have different color balance or smoother color distribution.
RGB histograms	Eight-bin histogram for each color channel.	Captures whether colors are concentrated, flat, or unusually distributed.
Grayscale histogram	Sixteen-bin brightness distribution.	Summarizes contrast and lighting patterns.
Edge-strength statistics	Mean, standard deviation, 10th percentile, and 90th percentile of gradient magnitude.	Generated images can differ in sharpness and fine edge consistency.

Feature group	What it measures	Why it may help
Noise residual statistics	Difference between grayscale image and a blurred version.	Real camera photos often contain sensor/compression noise that synthetic images may smooth out.
Frequency energy ratios	Low, mid, high Fourier energy and high/low ratio.	AI images can show different texture frequency patterns.
Blockiness indicators	Average edge changes along 8-pixel grid boundaries.	Detects JPEG-like or generator/compression artifacts.

Logistic Regression and ResNet18.

Logistic Regression uses the same handcrafted features but learns one linear decision boundary, so it is easier to interpret but less flexible. ResNet18 is a transfer-learning baseline: it starts from ImageNet-pretrained weights, replaces the final layer with a two-class classifier, and is fine-tuned for 10 epochs. In the demo, ResNet18 is not the final predictor; it is kept for the Grad-CAM-style attention image.

3 Validation and Results

All thresholds are selected using validation data only. The held-out test set is evaluated once after model and threshold choices are fixed. This prevents test-set leakage. On Tiny GenImage, the Random Forest threshold selected on validation is 0.545.

Table 3. Tiny GenImage 5k held-out test comparison.

Model	Type	Accuracy	Precision	Recall	F1	ROC-AUC
Random Forest with handcrafted features	Classical ML	98.2%	0.967	0.998	0.982	1.000
Logistic Regression with handcrafted features	Classical ML	78.9%	0.717	0.956	0.819	0.759
ResNet18 transfer learning, 10 epochs	Transfer learning	78.5%	0.755	0.844	0.797	0.865

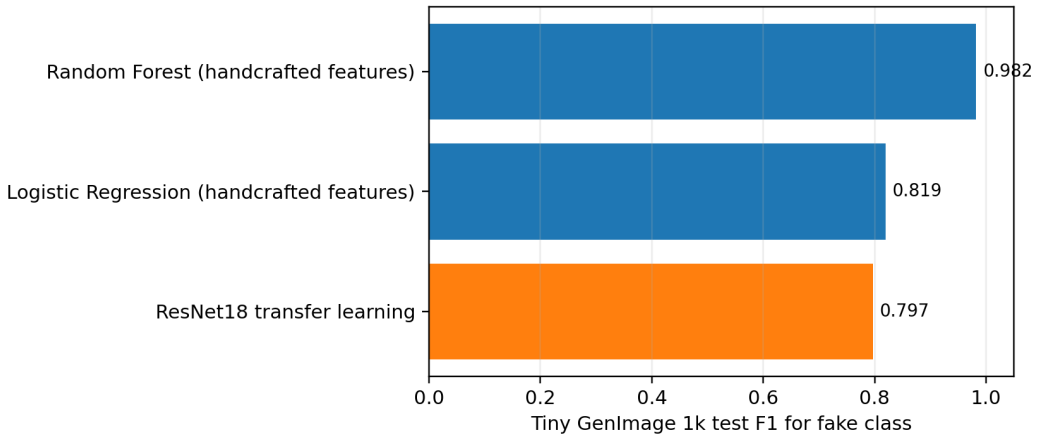

Figure 1. Tiny GenImage 5k F1 comparison. Random Forest performs best on this subset.

The smaller local dataset gives a similar lesson: classical handcrafted-feature models are strong, while a validation-weighted transfer-learning ensemble reaches 96.7% accuracy and 0.933 AI-class F1. I treat this as supporting evidence, not as the main deployed result.

4 Interpretation and Limitations

The Random Forest result does not mean the detector has solved AI-image detection. It means that this dataset contains low-level signals that the handcrafted features capture well. The model may rely on dataset-specific artifacts such as compression style, resolution patterns, generator texture, or repeated preprocessing. These signals can be useful for a course project, but they may not transfer to every real-world image source.

A key limitation is generalization to newer AI generators and image-to-image systems. If a new model produces images with more realistic camera noise, different compression, or fewer frequency artifacts, the Random Forest may perform worse. It may also struggle with edited real photos, screenshots, low-resolution uploads, or image-to-image

outputs where a real photo is partially modified. In a quick qualitative check, image-to-image style examples such as the user's 'image2' case were less reliable, which is consistent with the limitation that the model learned Tiny GenImage patterns rather than universal proof of image origin.

For this reason, the Hugging Face Space presents the output as statistical decision support, not proof. False positives may unfairly label real images as fake, while false negatives may miss synthetic content. A real deployment would need larger multi-generator training data, uncertainty thresholds, periodic retraining, and human review.

5 Deployment and Conclusion

The deployed Hugging Face Space uses the Tiny GenImage Random Forest model for final prediction and the ResNet18 model only for the attention visualization. This design matches the report result while still giving an interpretable visual bonus. Overall, the project demonstrates supervised learning, stratified splitting, feature engineering, validation-based threshold selection, model comparison, error analysis, and deployment. Readers who want the complete project files, result artifacts, and model files can visit the GitHub repository linked on the first page.

Appendix Code

tiny_genimage_5k_compare.py

```
import io
import json
import random
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pyarrow.parquet as pq
import requests
import torch
import torch.nn as nn
import torch.optim as optim
from PIL import Image, ImageFilter
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader, Dataset
from torchvision import models, transforms
from torchvision.models import ResNet18_Weights
from tqdm.auto import tqdm


SEED = 142
DATASET = "TheKernel01/Tiny-GenImage"
ROOT = Path(r"G:\CodexProjects\New project 3")
OUT_DIR = ROOT / "results" / "tiny_genimage_5k"
CACHE_DIR = ROOT / "data_tiny_genimage_5k" / "parquet"
LABEL_TO_NAME = {0: "real", 1: "fake"}
AI_CLASS_ID = 1


def seed_everything():
    random.seed(SEED)
    np.random.seed(SEED)
    torch.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)
    torch.backends.cudnn.benchmark = True


def get_parquet_files():
    url = f"https://datasets-server.huggingface.co/parquet?dataset={DATASET}"
    data = requests.get(url, timeout=120).json()["parquet_files"]
    files = {}
    for row in data:
        files.setdefault(row["split"], []).append(row)
    for split in files:
        files[split] = sorted(files[split], key=lambda x: x["filename"])
    return files


def download_file(url: str, path: Path):
    if path.exists() and path.stat().st_size &gt; 0:
        return
    path.parent.mkdir(parents=True, exist_ok=True)
    with requests.get(url, stream=True, timeout=240) as response:
        response.raise_for_status()
        total = int(response.headers.get("content-length", 0))
        with path.open("wb") as f, tqdm(total=total, unit="B", unit_scale=True, desc=path.name) as bar:
            for chunk in response.iter_content(chunk_size=1024 * 1024):
                if chunk:
                    f.write(chunk)
                    bar.update(len(chunk))


def image_from_cell(cell):
    if isinstance(cell, dict):
        if cell.get("bytes") is not None:
            return Image.open(io.BytesIO(cell["bytes"])).convert("RGB")
        if cell.get("path"):
            return Image.open(cell["path"]).convert("RGB")
    if hasattr(cell, "as_py"):
        return image_from_cell(cell.as_py())
    raise ValueError(f"Unsupported image cell type: {type(cell)}")


def read_rows(split: str, n_rows: int, shard_count: int):
    files = get_parquet_files()[split][:shard_count]
    frames = []
    for file_info in files:
        local_path = CACHE_DIR / split / file_info["filename"]
        download_file(file_info["url"], local_path)
        table = pq.read_table(local_path, columns=["image", "label", "generator"])
        frames.append(table.to_pandas())
        if sum(len(frame) for frame in frames) &gt;= n_rows:
            break
    df = pd.concat(frames, ignore_index=True).iloc[:n_rows].copy()
    df["source_split"] = split
    df["label_name"] = df["label"].map(LABEL_TO_NAME)
    return df
def safe_stats(values):
    values = np.asarray(values, dtype=np.float32).ravel()
```

```
    if values.size == 0:
        return [0.0, 0.0, 0.0, 0.0]
    return [
        float(np.mean(values)),
        float(np.std(values)),
        float(np.percentile(values, 10)),
        float(np.percentile(values, 90)),
    ]


def handcrafted_features(image_cell):
    image = image_from_cell(image_cell)
    width, height = image.size
    small_rgb = image.resize((128, 128), Image.Resampling.BILINEAR)
    arr = np.asarray(small_rgb).astype(np.float32) / 255.0
    gray = np.asarray(small_rgb.convert("L")).astype(np.float32) / 255.0

    features = [width / max(height, 1), np.log1p(width * height)]
    features.extend(arr.mean(axis=(0, 1)).tolist())
    features.extend(arr.std(axis=(0, 1)).tolist())
    for channel in range(3):
        hist, _ = np.histogram(arr[:, :, channel], bins=8, range=(0.0, 1.0), density=True)
        features.extend(hist.astype(float).tolist())
    gray_hist, _ = np.histogram(gray, bins=16, range=(0.0, 1.0), density=True)
    features.extend(gray_hist.astype(float).tolist())

    gx = np.diff(gray, axis=1, append=gray[:, -1:])
    gy = np.diff(gray, axis=0, append=gray[-1:, :])
    grad_mag = np.sqrt(gx * gx + gy * gy)
    features.extend(safe_stats(grad_mag))

    blurred = np.asarray(small_rgb.convert("L").filter(ImageFilter.BoxBlur(1))).astype(np.float32) / 255.0
    features.extend(safe_stats(np.abs(gray - blurred)))

    spectrum = np.abs(np.fft.fftshift(np.fft.fft2(gray)))
    yy, xx = np.indices(spectrum.shape)
    center_y, center_x = (np.array(spectrum.shape) - 1) / 2
    radius = np.sqrt((yy - center_y) ** 2 + (xx - center_x) ** 2)
    total_energy = spectrum.sum() + 1e-8
    low = spectrum[radius &lt; 12].sum() / total_energy
    mid = spectrum[(radius &gt;= 12) &amp; (radius &lt; 36)].sum() / total_energy
    high = spectrum[radius &gt;= 36].sum() / total_energy
    features.extend([float(low), float(mid), float(high), float(high / (low + 1e-8))])

    vertical_edges = np.abs(np.diff(gray, axis=1))
    horizontal_edges = np.abs(np.diff(gray, axis=0))
    block_v = vertical_edges[:, 7::8].mean() if vertical_edges[:, 7::8].size else 0.0
    block_h = horizontal_edges[7::8, :].mean() if horizontal_edges[7::8, :].size else 0.0
    features.extend([float(block_v), float(block_h), float((block_v + block_h) / (grad_mag.mean() + 1e-8))])
    return np.asarray(features, dtype=np.float32)


def metric_dict(y_true, prob_ai, threshold=0.5):
    y_true = np.asarray(y_true).astype(int)
    prob_ai = np.asarray(prob_ai).astype(float)
    pred = (prob_ai &gt;= threshold).astype(int)
    return {
        "threshold": float(threshold),
        "accuracy": float(accuracy_score(y_true, pred)),
        "precision_ai": float(precision_score(y_true, pred, zero_division=0)),
        "recall_ai": float(recall_score(y_true, pred, zero_division=0)),
        "f1_ai": float(f1_score(y_true, pred, zero_division=0)),
        "roc_auc": float(roc_auc_score(y_true, prob_ai)) if len(np.unique(y_true)) == 2 else float("nan"),
    }


def best_threshold(y_true, prob_ai):
    thresholds = np.linspace(0.05, 0.95, 181)
    return float(max(thresholds, key=lambda t: f1_score(y_true, prob_ai &gt;= t, zero_division=0)))


class TinyImageDataset(Dataset):
    def __init__(self, frame, transform):
        self.frame = frame.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.frame)

    def __getitem__(self, idx):
        row = self.frame.iloc[idx]
        image = image_from_cell(row["image"])
        return self.transform(image), int(row["label"])


def evaluate_resnet(model, loader, device):
    model.eval()
    y_true, prob_ai = [], []
    with torch.no_grad():
        for images, labels in tqdm(loader, desc="ResNet18 eval"):
            images = images.to(device, non_blocking=True)
            probs = torch.softmax(model(images), dim=1)[:, AI_CLASS_ID].detach().cpu().numpy()
            y_true.extend(labels.numpy().tolist())
            prob_ai.extend(probs.tolist())
    return np.asarray(y_true), np.asarray(prob_ai)


def train_resnet(train_df, val_df, test_df):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print("Device:", device)
    if torch.cuda.is_available():
        print("GPU:", torch.cuda.get_device_name(0))

    train_transform = transforms.Compose(
        [
            transforms.Resize((256, 256)),
            transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
```

```
        ]
    )
    eval_transform = transforms.Compose(
        [
            transforms.Resize((256, 256)),
            transforms.CenterCrop(224),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
        ]
    )
    train_loader = DataLoader(TinyImageDataset(train_df, train_transform), batch_size=32, shuffle=True, num_workers=0,
 pin_memory=True)
    val_loader = DataLoader(TinyImageDataset(val_df, eval_transform), batch_size=32, shuffle=False, num_workers=0,
pin_memory=True)
    test_loader = DataLoader(TinyImageDataset(test_df, eval_transform), batch_size=32, shuffle=False, num_workers=0,
pin_memory=True)

    model = models.resnet18(weights=ResNet18_Weights.DEFAULT)
    model.fc = nn.Linear(model.fc.in_features, 2)
    model = model.to(device)

    counts = train_df["label"].value_counts().reindex([0, 1]).fillna(0).astype(float).values
    class_weights = torch.tensor(counts.sum() / (2 * np.maximum(counts, 1)), dtype=torch.float32, device=device)
    criterion = nn.CrossEntropyLoss(weight=class_weights)
    optimizer = optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-4)
    scaler = torch.amp.GradScaler("cuda", enabled=device.type == "cuda")

    best_state = None
    best_val_f1 = -1
    history = []
    for epoch in range(1, 11):
        model.train()
        total_loss = 0.0
        seen = 0
        for images, labels in tqdm(train_loader, desc=f"ResNet18 epoch {epoch}/10"):
            images = images.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)
            optimizer.zero_grad(set_to_none=True)
            with torch.amp.autocast("cuda", enabled=device.type == "cuda"):
                outputs = model(images)
                loss = criterion(outputs, labels)
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
            total_loss += loss.item() * labels.size(0)
            seen += labels.size(0)

        y_val, p_val = evaluate_resnet(model, val_loader, device)
        threshold = best_threshold(y_val, p_val)
        metrics = metric_dict(y_val, p_val, threshold)
        metrics.update({"epoch": epoch, "train_loss": total_loss / max(seen, 1)})
        history.append(metrics)
        print(f"epoch {epoch}: loss={metrics['train_loss']:.4f}, val_f1={metrics['f1_ai']:.4f}, val_auc={metrics[
'roc_auc']:.4f}, thr={threshold:.3f}")
        if metrics["f1_ai"] &gt; best_val_f1:
            best_val_f1 = metrics["f1_ai"]
            best_state = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}

    model.load_state_dict(best_state)
    model.to(device)
    y_val, p_val = evaluate_resnet(model, val_loader, device)
    threshold = best_threshold(y_val, p_val)
    y_test, p_test = evaluate_resnet(model, test_loader, device)
    torch.save(model.state_dict(), OUT_DIR / "tiny_genimage_5k_resnet18_best.pt")
    return pd.DataFrame(history), metric_dict(y_test, p_test, threshold), threshold


def main():
    seed_everything()
    OUT_DIR.mkdir(parents=True, exist_ok=True)

    train_pool = read_rows("train", n_rows=4000, shard_count=2)
    test_df = read_rows("validation", n_rows=1000, shard_count=1)
    fit_idx, val_idx = train_test_split(
        np.arange(len(train_pool)),
        test_size=400,
        random_state=SEED,
        stratify=train_pool["label"],
    )
    fit_df = train_pool.iloc[fit_idx].reset_index(drop=True)
    val_df = train_pool.iloc[val_idx].reset_index(drop=True)
    test_df = test_df.reset_index(drop=True)

    split_summary = {
        "fit": fit_df["label_name"].value_counts().to_dict(),
        "validation": val_df["label_name"].value_counts().to_dict(),
        "test": test_df["label_name"].value_counts().to_dict(),
    }
    print("Split summary:", split_summary)

    frames = []
    for name, frame in [("fit", fit_df), ("validation", val_df), ("test", test_df)]:
        tmp = frame[["label", "label_name", "generator"]].copy()
        tmp["split"] = name
        frames.append(tmp)
    pd.concat(frames, ignore_index=True).to_csv(OUT_DIR / "tiny_genimage_5k_splits.csv", index=False)

    feature_sets = {}
    for split, frame in [("fit", fit_df), ("validation", val_df), ("test", test_df)]:
        x = np.vstack([handcrafted_features(cell) for cell in tqdm(frame["image"], desc=f"Handcrafted features: {
split}")])
        y = frame["label"].to_numpy(dtype=int)
        feature_sets[split] = (x, y)

    x_fit, y_fit = feature_sets["fit"]
    x_val, y_val = feature_sets["validation"]
    x_test, y_test = feature_sets["test"]

    model_specs = {
        "Logistic Regression (handcrafted features)": make_pipeline(
```

```
            StandardScaler(),
            LogisticRegression(max_iter=3000, class_weight="balanced", random_state=SEED),
        ),
        "Random Forest (handcrafted features)": RandomForestClassifier(
            n_estimators=400,
            min_samples_leaf=2,
            max_features="sqrt",
            class_weight="balanced",
            random_state=SEED,
            n_jobs=-1,
        ),
    }

    rows = []
    for model_name, model in model_specs.items():
        model.fit(x_fit, y_fit)
        val_prob = model.predict_proba(x_val)[:, AI_CLASS_ID]
        threshold = best_threshold(y_val, val_prob)
        test_prob = model.predict_proba(x_test)[:, AI_CLASS_ID]
        metrics = metric_dict(y_test, test_prob, threshold)
        metrics.update({"model": model_name, "model_family": "Classical ML", "split": "test"})
        rows.append(metrics)
        print(model_name, metrics)

    history, resnet_metrics, resnet_threshold = train_resnet(fit_df, val_df, test_df)
    history.to_csv(OUT_DIR / "tiny_genimage_5k_resnet18_history.csv", index=False)
    resnet_metrics.update({"model": "ResNet18 transfer learning", "model_family": "Transfer Learning", "split":
"test"})
    rows.append(resnet_metrics)
    print("ResNet18 transfer learning", resnet_metrics)

    results = pd.DataFrame(rows)[
        ["model_family", "model", "split", "threshold", "accuracy", "precision_ai", "recall_ai", "f1_ai", "roc_auc"]
    ].sort_values("f1_ai", ascending=False)
    results.to_csv(OUT_DIR / "tiny_genimage_5k_model_comparison.csv", index=False)

    colors = {"Classical ML": "#1f77b4", "Transfer Learning": "#ff7f0e"}
    plt.figure(figsize=(7.6, 3.4))
    plt.barh(results["model"], results["f1_ai"], color=[colors.get(x, "#2ca02c") for x in results["model_family"]])
    plt.xlabel("Tiny GenImage 1k test F1 for fake class")
    plt.xlim(0, 1.05)
    plt.gca().invert_yaxis()
    plt.grid(axis="x", alpha=0.25)
    for idx, (_, row) in enumerate(results.iterrows()):
        plt.text(min(row["f1_ai"] + 0.01, 1.02), idx, f"{row['f1_ai']:.3f}", va="center", fontsize=9)
    plt.tight_layout()
    plt.savefig(OUT_DIR / "tiny_genimage_5k_test_f1.png", dpi=220)
    plt.close()

    summary = {
        "dataset": DATASET,
        "design": "4000 train-pool rows from train split, internally split into 3600 fit and 400 validation; 1000
held-out rows from validation split used as test.",
        "split_summary": split_summary,
        "results": results.to_dict(orient="records"),
    }
    (OUT_DIR / "tiny_genimage_5k_summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
    print("\nFinal Tiny GenImage 5k comparison:")
    print(results.to_string(index=False))
    print("\nWrote results to", OUT_DIR)


if __name__ == "__main__":
    main()
```

hf_space_ai_detector\app.py

```
from pathlib import Path
import json

import gradio as gr
import joblib
import numpy as np
from PIL import Image, ImageFilter
import torch
import torch.nn as nn
from torchvision import models, transforms


ROOT = Path(__file__).resolve().parent
MODEL_DIR = ROOT / "models"
IMAGE_SIZE = 224
AI_CLASS_ID = 1
ID_TO_LABEL = {0: "Real", 1: "AI-generated"}
RF_MODEL_LABEL = "Random Forest with handcrafted features"
RESNET_MODEL_NAME = "tiny_genimage_resnet18"
RESNET_MODEL_LABEL = "Tiny GenImage ResNet18, 10 epochs"

with (MODEL_DIR / "tiny_genimage_random_forest_metadata.json").open("r", encoding="utf-8") as f:
    RF_METADATA = json.load(f)

RF_THRESHOLD = float(RF_METADATA["threshold"])
RF_METRICS = RF_METADATA["metrics"]
RESNET_METRICS = {
    "test_accuracy": 0.785,
    "test_precision_fake": 0.7549,
    "test_recall_fake": 0.844,
    "test_f1_fake": 0.797,
    "test_roc_auc": 0.865,
}

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

LOCAL_COMPARISON_ROWS = [
    ["Logistic Regression + handcrafted features", "Classical ML", "98.7%", "0.974", "1.000"],
    ["Random Forest + handcrafted features", "Classical ML", "98.7%", "0.974", "1.000"],
    ["Validation-weighted ensemble", "Local comparison", "96.7%", "0.933", "0.993"],
    ["ResNet18", "Transfer learning", "96.0%", "0.919", "0.995"],
```

```
]

TINY_GENIMAGE_ROWS = [
    ["Random Forest + handcrafted features", "Classical ML", "98.2%", "0.982", "1.000"],
    ["Logistic Regression + handcrafted features", "Classical ML", "78.9%", "0.819", "0.759"],
    ["ResNet18, 10 epochs", "Transfer learning", "78.5%", "0.797", "0.865"],
]

PROJECT_LINKS = """
- GitHub appendix: https://github.com/Theo9598/ai-vs-real-image-detector
- Final report: see `reports/142A_final_project_ai_detector_submission.pdf` in GitHub
- Tiny GenImage dataset: https://huggingface.co/datasets/TheKernel01/Tiny-GenImage
"""

EVAL_TRANSFORM = transforms.Compose(
    [
        transforms.Resize((256, 256)),
        transforms.CenterCrop(IMAGE_SIZE),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ]
)


def make_model(name: str) -&gt; nn.Module:
    if name == RESNET_MODEL_NAME:
        model = models.resnet18(weights=None)
        model.fc = nn.Linear(model.fc.in_features, 2)
        return model
    raise ValueError(f"Unknown model: {name}")


def load_models() -&gt; dict[str, nn.Module]:
    resnet = make_model(RESNET_MODEL_NAME)
    state = torch.load(MODEL_DIR / f"{RESNET_MODEL_NAME}_best.pt", map_location=DEVICE)
    resnet.load_state_dict(state)
    resnet.to(DEVICE)
    resnet.eval()
    return {
        "random_forest": joblib.load(MODEL_DIR / "tiny_genimage_random_forest.joblib"),
        RESNET_MODEL_NAME: resnet,
    }


MODELS = load_models()


def safe_stats(values):
    values = np.asarray(values, dtype=np.float32).ravel()
    if values.size == 0:
        return [0.0, 0.0, 0.0, 0.0]
    return [
        float(np.mean(values)),
        float(np.std(values)),
        float(np.percentile(values, 10)),
        float(np.percentile(values, 90)),
    ]


def handcrafted_features(image: Image.Image) -&gt; np.ndarray:
    image = image.convert("RGB")
    width, height = image.size
    small_rgb = image.resize((128, 128), Image.Resampling.BILINEAR)
    arr = np.asarray(small_rgb).astype(np.float32) / 255.0
    gray = np.asarray(small_rgb.convert("L")).astype(np.float32) / 255.0

    features = [width / max(height, 1), np.log1p(width * height)]
    features.extend(arr.mean(axis=(0, 1)).tolist())
    features.extend(arr.std(axis=(0, 1)).tolist())
    for channel in range(3):
        hist, _ = np.histogram(arr[:, :, channel], bins=8, range=(0.0, 1.0), density=True)
        features.extend(hist.astype(float).tolist())
    gray_hist, _ = np.histogram(gray, bins=16, range=(0.0, 1.0), density=True)
    features.extend(gray_hist.astype(float).tolist())

    gx = np.diff(gray, axis=1, append=gray[:, -1:])
    gy = np.diff(gray, axis=0, append=gray[-1:, :])
    grad_mag = np.sqrt(gx * gx + gy * gy)
    features.extend(safe_stats(grad_mag))

    blurred = np.asarray(small_rgb.convert("L").filter(ImageFilter.BoxBlur(1))).astype(np.float32) / 255.0
    features.extend(safe_stats(np.abs(gray - blurred)))

    spectrum = np.abs(np.fft.fftshift(np.fft.fft2(gray)))
    yy, xx = np.indices(spectrum.shape)
    center_y, center_x = (np.array(spectrum.shape) - 1) / 2
    radius = np.sqrt((yy - center_y) ** 2 + (xx - center_x) ** 2)
    total_energy = spectrum.sum() + 1e-8
    low = spectrum[radius &lt; 12].sum() / total_energy
    mid = spectrum[(radius &gt;= 12) &amp;amp; (radius &lt; 36)].sum() / total_energy
    high = spectrum[radius &gt;= 36].sum() / total_energy
    features.extend([float(low), float(mid), float(high), float(high / (low + 1e-8))])

    vertical_edges = np.abs(np.diff(gray, axis=1))
    horizontal_edges = np.abs(np.diff(gray, axis=0))
    block_v = vertical_edges[:, 7::8].mean() if vertical_edges[:, 7::8].size else 0.0
    block_h = horizontal_edges[7::8, :].mean() if horizontal_edges[7::8, :].size else 0.0
    features.extend([float(block_v), float(block_h), float((block_v + block_h) / (grad_mag.mean() + 1e-8))])
    return np.asarray(features, dtype=np.float32)


def resnet_probability(image: Image.Image) -&gt; float:
    image = image.convert("RGB")
    x = EVAL_TRANSFORM(image).unsqueeze(0).to(DEVICE)
    with torch.no_grad():
        output = MODELS[RESNET_MODEL_NAME](x)
        prob_ai = torch.softmax(output, dim=1)[0, AI_CLASS_ID].item()
    return float(prob_ai)
def predict_probabilities(image: Image.Image) -&gt; tuple[float, dict[str, float]]:
    features = handcrafted_features(image).reshape(1, -1)
```

```
    rf_prob = float(MODELS["random_forest"].predict_proba(features)[0, AI_CLASS_ID])
    resnet_prob = resnet_probability(image)
    return rf_prob, {
        RF_MODEL_LABEL: rf_prob,
        f"{RESNET_MODEL_LABEL} attention model": resnet_prob,
    }


def resnet_attention(image: Image.Image) -&gt; Image.Image:
    """Simple Grad-CAM-style attention overlay for the AI class using ResNet18."""
    model = MODELS[RESNET_MODEL_NAME]
    layer = model.layer4[-1]
    activations = []
    gradients = []

    def forward_hook(_module, _inputs, output):
        activations.append(output)

    def backward_hook(_module, _grad_input, grad_output):
        gradients.append(grad_output[0])

    handle_fwd = layer.register_forward_hook(forward_hook)
    handle_bwd = layer.register_full_backward_hook(backward_hook)

    try:
        rgb = image.convert("RGB")
        x = EVAL_TRANSFORM(rgb).unsqueeze(0).to(DEVICE)
        model.zero_grad(set_to_none=True)
        output = model(x)
        score = output[:, AI_CLASS_ID].sum()
        score.backward()

        act = activations[0].detach()
        grad = gradients[0].detach()
        weights = grad.mean(dim=(2, 3), keepdim=True)
        cam = (weights * act).sum(dim=1).squeeze()
        cam = torch.relu(cam)
        cam = cam - cam.min()
        cam = cam / (cam.max() + 1e-8)
        cam = cam.cpu().numpy()

        cam_img = Image.fromarray(np.uint8(cam * 255)).resize((IMAGE_SIZE, IMAGE_SIZE), Image.BILINEAR)
        heat = np.asarray(cam_img).astype(np.float32) / 255.0
        base = np.asarray(rgb.resize((IMAGE_SIZE, IMAGE_SIZE))).astype(np.float32) / 255.0

        overlay = base.copy()
        overlay[..., 0] = np.clip(0.55 * base[..., 0] + 0.45 * heat, 0, 1)
        overlay[..., 1] = np.clip(0.65 * base[..., 1], 0, 1)
        overlay[..., 2] = np.clip(0.65 * base[..., 2], 0, 1)
        return Image.fromarray(np.uint8(overlay * 255))
    finally:
        handle_fwd.remove()
        handle_bwd.remove()


def analyze(image: Image.Image):
    if image is None:
        return "Upload an image to run the detector.", {}, None

    prob_ai, per_model = predict_probabilities(image)
    pred_id = int(prob_ai &gt;= RF_THRESHOLD)
    verdict = ID_TO_LABEL[pred_id]
    confidence = prob_ai if pred_id == AI_CLASS_ID else 1.0 - prob_ai
    attention = resnet_attention(image)

    text = f"""
### Prediction: {verdict}

**AI probability:** {prob_ai:.4f}
**Decision threshold:** {RF_THRESHOLD:.4f}
**Displayed confidence:** {confidence:.4f}
**Final predictor:** {RF_MODEL_LABEL}

This is the highest-scoring Tiny GenImage 5k model from the report. Its held-out test results were accuracy {
RF_METRICS["accuracy"]:.3f}, F1 {RF_METRICS["f1_ai"]:.3f}, and ROC-AUC {RF_METRICS["roc_auc"]:.3f}.

This is a statistical detector, not proof of image origin. The heatmap is generated by the ResNet18 comparison model
as an attention visualization, not by the Random Forest itself.
"""
    rounded = {name: round(value, 4) for name, value in per_model.items()}
    return text, rounded, attention


CSS = """
.main-title {max-width: 980px; margin: 0 auto 10px auto;}
.note-box {
    border-left: 4px solid #4b5563;
    background: #f8fafc;
    padding: 12px 14px;
    border-radius: 4px;
}
"""


with gr.Blocks(title="AI vs Real Image Detector", theme=gr.themes.Soft(), css=CSS) as demo:
    gr.Markdown(
        """
&lt;div class="main-title"&gt;

# AI vs Real Image Detector

IEOR 142A course project demo. Upload an image to estimate whether it is AI-generated.
The final detector uses the Tiny GenImage 5k Random Forest report model.
The output is statistical decision support, not proof of image origin.

&lt;/div&gt;
"""
    )

    with gr.Tabs():
```

```
        with gr.Tab("Detector"):
            with gr.Row():
                with gr.Column(scale=1):
                    image_input = gr.Image(type="pil", label="Upload Image")
                    analyze_button = gr.Button("Analyze", variant="primary")
                with gr.Column(scale=1):
                    result_text = gr.Markdown(label="Final Result")
                    model_json = gr.JSON(label="Model AI Probabilities")
                    attention_image = gr.Image(label="ResNet18 Attention Regions")

            analyze_button.click(
                fn=analyze,
                inputs=image_input,
                outputs=[result_text, model_json, attention_image],
            )

        with gr.Tab("Results"):
            gr.Markdown(
                """
## Model comparison

The deployed detector uses the Tiny GenImage 5k Random Forest model from the report because it has the best held-out
test F1.
The ResNet18 model is still used for the attention visualization in the Detector tab.
"""
            )
            gr.Markdown("### Local project dataset, held-out test")
            gr.Dataframe(
                value=LOCAL_COMPARISON_ROWS,
                headers=["Model", "Type", "Accuracy", "F1, AI/fake", "ROC-AUC"],
                interactive=False,
                wrap=True,
            )
            gr.Markdown("### Tiny GenImage 5k subset, held-out test")
            gr.Dataframe(
                value=TINY_GENIMAGE_ROWS,
                headers=["Model", "Type", "Accuracy", "F1, fake", "ROC-AUC"],
                interactive=False,
                wrap=True,
            )

        with gr.Tab("Interpretation"):
            gr.Markdown(
                """
## What the project learned

The main lesson is not simply that one model had the highest accuracy. Classical machine learning
baselines were very strong when images were represented with handcrafted features such as color,
edge, noise, frequency, and blockiness statistics.

On the Tiny GenImage 5k subset, Random Forest outperformed a 10-epoch ResNet18 fine-tuning run.
This suggests that low-level image artifacts are highly informative in this dataset, and that model
complexity alone does not guarantee better performance.

&lt;div class="note-box"&gt;
The model-attention image shows regions that influenced the ResNet18 AI-class score. It should not
be interpreted as proof that a region is fake or AI-generated.
&lt;/div&gt;
"""
            )

        with gr.Tab("Links"):
            gr.Markdown(PROJECT_LINKS)


if __name__ == "__main__":
    demo.launch()
```

compare_course_vs_transfer.py

```
import json
import random
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from PIL import Image, ImageFilter
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader, Dataset
from torchvision import models, transforms
from torchvision.models import (
    EfficientNet_B0_Weights,
    MobileNet_V3_Small_Weights,
    ResNet18_Weights,
    ViT_B_16_Weights,
)
from tqdm.auto import tqdm


SEED = 142
```

```
DATA_ROOT = Path(r"G:\CodexProjects\New project 3\data")
RESULTS_DIR = Path(r"G:\CodexProjects\New project 3\results")
IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".webp", ".bmp"}
LABEL_TO_ID = {"real": 0, "ai": 1}
ID_TO_LABEL = {0: "real", 1: "ai"}
AI_CLASS_ID = 1


def seed_everything():
    random.seed(SEED)
    np.random.seed(SEED)
    torch.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)
    torch.backends.cudnn.benchmark = True


def collect_records(root: Path, label_name: str) -&gt; list[dict]:
    rows = []
    for path in root.rglob("*"):
        if path.is_file() and path.suffix.lower() in IMAGE_EXTENSIONS:
            rows.append(
                {
                    "path": str(path),
                    "label": LABEL_TO_ID[label_name],
                    "label_name": label_name,
                    "category": path.parent.name,
                }
            )
    return rows


def load_or_create_splits() -&gt; pd.DataFrame:
    split_path = RESULTS_DIR / "dataset_splits.csv"
    if split_path.exists():
        df = pd.read_csv(split_path)
        needed = {"path", "label", "label_name", "category", "split"}
        if needed.issubset(df.columns):
            return df

    records = collect_records(DATA_ROOT / "Ai_generated_dataset", "ai")
    records += collect_records(DATA_ROOT / "real_dataset", "real")
    df = pd.DataFrame(records).drop_duplicates("path").reset_index(drop=True)
    indices = np.arange(len(df))
    train_val_idx, test_idx = train_test_split(
        indices, test_size=0.15, random_state=SEED, stratify=df["label"]
    )
    train_idx, val_idx = train_test_split(
        train_val_idx,
        test_size=0.15 / 0.85,
        random_state=SEED,
        stratify=df.loc[train_val_idx, "label"],
    )
    df["split"] = "unused"
    df.loc[train_idx, "split"] = "train"
    df.loc[val_idx, "split"] = "validation"
    df.loc[test_idx, "split"] = "test"
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    df.to_csv(split_path, index=False)
    return df


def safe_stats(values: np.ndarray) -&gt; list[float]:
    values = values.astype(np.float32).ravel()
    if values.size == 0:
        return [0.0, 0.0, 0.0, 0.0]
    return [
        float(np.mean(values)),
        float(np.std(values)),
        float(np.percentile(values, 10)),
        float(np.percentile(values, 90)),
    ]


def handcrafted_features(path: str) -&gt; np.ndarray:
    image = Image.open(path).convert("RGB")
    width, height = image.size
    small_rgb = image.resize((128, 128), Image.Resampling.BILINEAR)
    arr = np.asarray(small_rgb).astype(np.float32) / 255.0
    gray = np.asarray(small_rgb.convert("L")).astype(np.float32) / 255.0

    features = []
    features.extend([width / max(height, 1), np.log1p(width * height)])

    # Color statistics.
    features.extend(arr.mean(axis=(0, 1)).tolist())
    features.extend(arr.std(axis=(0, 1)).tolist())
    for channel in range(3):
        hist, _ = np.histogram(arr[:, :, channel], bins=8, range=(0.0, 1.0), density=True)
        features.extend(hist.astype(float).tolist())

    gray_hist, _ = np.histogram(gray, bins=16, range=(0.0, 1.0), density=True)
    features.extend(gray_hist.astype(float).tolist())

    # Edge and texture statistics from simple gradients.
    gx = np.diff(gray, axis=1, append=gray[:, -1:])
    gy = np.diff(gray, axis=0, append=gray[-1:, :])
    grad_mag = np.sqrt(gx * gx + gy * gy)
    features.extend(safe_stats(grad_mag))

    # Noise residual: original grayscale minus a small box blur.
    blurred = np.asarray(small_rgb.convert("L").filter(ImageFilter.BoxBlur(1))).astype(np.float32) / 255.0
    residual = gray - blurred
    features.extend(safe_stats(np.abs(residual)))

    # Frequency energy bands.
    spectrum = np.abs(np.fft.fftshift(np.fft.fft2(gray)))
    yy, xx = np.indices(spectrum.shape)
    center_y, center_x = (np.array(spectrum.shape) - 1) / 2
    radius = np.sqrt((yy - center_y) ** 2 + (xx - center_x) ** 2)
    total_energy = spectrum.sum() + 1e-8
```

```
    low = spectrum[radius &lt; 12].sum() / total_energy
    mid = spectrum[(radius &gt;= 12) &amp; (radius &lt; 36)].sum() / total_energy
    high = spectrum[radius &gt;= 36].sum() / total_energy
    features.extend([float(low), float(mid), float(high), float(high / (low + 1e-8))])

    # JPEG/blockiness proxy.
    vertical_edges = np.abs(np.diff(gray, axis=1))
    horizontal_edges = np.abs(np.diff(gray, axis=0))
    block_v = vertical_edges[:, 7::8].mean() if vertical_edges[:, 7::8].size else 0.0
    block_h = horizontal_edges[7::8, :].mean() if horizontal_edges[7::8, :].size else 0.0
    features.extend([float(block_v), float(block_h), float((block_v + block_h) / (grad_mag.mean() + 1e-8))])

    return np.asarray(features, dtype=np.float32)


def metric_dict(y_true, prob_ai, threshold=0.5) -&gt; dict:
    y_true = np.asarray(y_true).astype(int)
    prob_ai = np.asarray(prob_ai).astype(float)
    y_pred = (prob_ai &gt;= threshold).astype(int)
    return {
        "threshold": float(threshold),
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "precision_ai": float(precision_score(y_true, y_pred, zero_division=0)),
        "recall_ai": float(recall_score(y_true, y_pred, zero_division=0)),
        "f1_ai": float(f1_score(y_true, y_pred, zero_division=0)),
        "roc_auc": float(roc_auc_score(y_true, prob_ai)) if len(np.unique(y_true)) == 2 else float("nan"),
    }


def best_threshold(y_true, prob_ai) -&gt; float:
    thresholds = np.linspace(0.05, 0.95, 181)
    return float(max(thresholds, key=lambda t: f1_score(y_true, prob_ai &gt;= t, zero_division=0)))


def train_course_models(df: pd.DataFrame):
    split_frames = {name: df[df["split"] == name].reset_index(drop=True) for name in ["train", "validation", "test"]}
    feature_cache = {}
    for split, frame in split_frames.items():
        x = np.vstack(
            [handcrafted_features(path) for path in tqdm(frame["path"], desc=f"Handcrafted features: {split}")]
        )
        y = frame["label"].to_numpy(dtype=int)
        feature_cache[split] = (x, y)

    x_train, y_train = feature_cache["train"]
    x_val, y_val = feature_cache["validation"]
    x_test, y_test = feature_cache["test"]

    models_to_fit = {
        "Logistic Regression (handcrafted features)": make_pipeline(
            StandardScaler(),
            LogisticRegression(max_iter=3000, class_weight="balanced", random_state=SEED),
        ),
        "Random Forest (handcrafted features)": RandomForestClassifier(
            n_estimators=400,
            min_samples_leaf=2,
            max_features="sqrt",
            class_weight="balanced",
            random_state=SEED,
            n_jobs=-1,
        ),
    }

    rows = []
    for name, model in models_to_fit.items():
        model.fit(x_train, y_train)
        val_prob = model.predict_proba(x_val)[:, AI_CLASS_ID]
        threshold = best_threshold(y_val, val_prob)
        val_metrics = metric_dict(y_val, val_prob, threshold)
        val_metrics.update({"model": name, "model_family": "Classical ML", "split": "validation"})
        rows.append(val_metrics)

        test_prob = model.predict_proba(x_test)[:, AI_CLASS_ID]
        test_metrics = metric_dict(y_test, test_prob, threshold)
        test_metrics.update({"model": name, "model_family": "Classical ML", "split": "test"})
        rows.append(test_metrics)

    return pd.DataFrame(rows)


class ImagePathDataset(Dataset):
    def __init__(self, frame: pd.DataFrame, transform):
        self.frame = frame.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.frame)

    def __getitem__(self, idx):
        row = self.frame.iloc[idx]
        image = Image.open(row["path"]).convert("RGB")
        return self.transform(image), int(row["label"])


def make_transfer_model(name: str):
    if name == "resnet18":
        model = models.resnet18(weights=ResNet18_Weights.DEFAULT)
        model.fc = nn.Linear(model.fc.in_features, 2)
        return model
    if name == "efficientnet_b0":
        model = models.efficientnet_b0(weights=EfficientNet_B0_Weights.DEFAULT)
        model.classifier[1] = nn.Linear(model.classifier[1].in_features, 2)
        return model
    if name == "mobilenet_v3_small":
        model = models.mobilenet_v3_small(weights=MobileNet_V3_Small_Weights.DEFAULT)
        model.classifier[3] = nn.Linear(model.classifier[3].in_features, 2)
        return model
    if name == "vit_b_16":
        model = models.vit_b_16(weights=ViT_B_16_Weights.DEFAULT)
        model.heads.head = nn.Linear(model.heads.head.in_features, 2)
```

```
        return model
    raise ValueError(name)


def evaluate_transfer_model(model, loader, device):
    model.eval()
    y_true, prob_ai = [], []
    with torch.no_grad():
        for images, labels in tqdm(loader, desc="Transfer inference"):
            images = images.to(device, non_blocking=True)
            probs = torch.softmax(model(images), dim=1)[:, AI_CLASS_ID].detach().cpu().numpy()
            prob_ai.extend(probs.tolist())
            y_true.extend(labels.numpy().tolist())
    return np.asarray(y_true), np.asarray(prob_ai)


def evaluate_transfer_models(df: pd.DataFrame):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print("Device:", device)
    if torch.cuda.is_available():
        print("GPU:", torch.cuda.get_device_name(0))

    eval_transform = transforms.Compose(
        [
            transforms.Resize((256, 256)),
            transforms.CenterCrop(224),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
        ]
    )
    val_loader = DataLoader(
        ImagePathDataset(df[df["split"] == "validation"], eval_transform),
        batch_size=16,
        shuffle=False,
        num_workers=0,
        pin_memory=True,
    )
    test_loader = DataLoader(
        ImagePathDataset(df[df["split"] == "test"], eval_transform),
        batch_size=16,
        shuffle=False,
        num_workers=0,
        pin_memory=True,
    )

    model_names = ["resnet18", "efficientnet_b0", "mobilenet_v3_small", "vit_b_16"]
    rows = []
    val_probs_by_model = {}
    test_probs_by_model = {}
    y_val_ref = y_test_ref = None

    for name in model_names:
        weight_path = RESULTS_DIR / f"{name}_best.pt"
        if not weight_path.exists():
            print(f"Skipping {name}: missing {weight_path}")
            continue
        model = make_transfer_model(name).to(device)
        model.load_state_dict(torch.load(weight_path, map_location=device))
        y_val, val_prob = evaluate_transfer_model(model, val_loader, device)
        threshold = best_threshold(y_val, val_prob)
        val_metrics = metric_dict(y_val, val_prob, threshold)
        val_metrics.update({"model": name, "model_family": "Transfer Learning", "split": "validation"})
        rows.append(val_metrics)

        y_test, test_prob = evaluate_transfer_model(model, test_loader, device)
        test_metrics = metric_dict(y_test, test_prob, threshold)
        test_metrics.update({"model": name, "model_family": "Transfer Learning", "split": "test"})
        rows.append(test_metrics)

        val_probs_by_model[name] = val_prob
        test_probs_by_model[name] = test_prob
        y_val_ref = y_val
        y_test_ref = y_test

        del model
        if torch.cuda.is_available():
            torch.cuda.empty_cache()

    if val_probs_by_model:
        order = list(val_probs_by_model)
        val_f1s = np.asarray(
            [metric_dict(y_val_ref, val_probs_by_model[name], best_threshold(y_val_ref, val_probs_by_model[name]))[
"f1_ai"] for name in order]
        )
        weights = (val_f1s + 1e-6) / (val_f1s.sum() + 1e-6 * len(val_f1s))
        val_ensemble = np.average(np.vstack([val_probs_by_model[name] for name in order]), axis=0, weights=weights)
        ensemble_threshold = best_threshold(y_val_ref, val_ensemble)
        val_metrics = metric_dict(y_val_ref, val_ensemble, ensemble_threshold)
        val_metrics.update({"model": "Validation-weighted ensemble", "model_family": "Local Ensemble", "split":
"validation"})
        rows.append(val_metrics)

        test_ensemble = np.average(np.vstack([test_probs_by_model[name] for name in order]), axis=0, weights=weights)
        test_metrics = metric_dict(y_test_ref, test_ensemble, ensemble_threshold)
        test_metrics.update({"model": "Validation-weighted ensemble", "model_family": "Local Ensemble", "split":
"test"})
        rows.append(test_metrics)

        cm = confusion_matrix(y_test_ref, test_ensemble &gt;= ensemble_threshold, labels=[0, 1])
        (RESULTS_DIR / "course_vs_transfer_ensemble_confusion_matrix.json").write_text(
            json.dumps({"labels": ["real", "ai"], "matrix": cm.tolist(), "weights": dict(zip(order, weights.tolist(
)))}, indent=2),
            encoding="utf-8",
        )

    return pd.DataFrame(rows)


def main():
    seed_everything()
```

```
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    df = load_or_create_splits()
    print("Dataset counts:", df["label_name"].value_counts().to_dict())
    print("Split counts:", df["split"].value_counts().to_dict())

    course_results = train_course_models(df)
    transfer_results = evaluate_transfer_models(df)
    all_results = pd.concat([course_results, transfer_results], ignore_index=True)
    all_results = all_results[
        ["model_family", "model", "split", "threshold", "accuracy", "precision_ai", "recall_ai", "f1_ai", "roc_auc"]
    ].sort_values(["split", "model_family", "f1_ai"], ascending=[True, True, False])

    all_results.to_csv(RESULTS_DIR / "course_vs_transfer_comparison.csv", index=False)
    all_results[all_results["split"] == "validation"].to_csv(
        RESULTS_DIR / "course_vs_transfer_validation.csv", index=False
    )
    all_results[all_results["split"] == "test"].to_csv(RESULTS_DIR / "course_vs_transfer_test.csv", index=False)

    test_results = all_results[all_results["split"] == "test"].copy()
    plt.figure(figsize=(9, 4.8))
    colors = {
        "Classical ML": "#6f6f6f",
        "Transfer Learning": "#2f2f2f",
        "Local Ensemble": "#000000",
    }
    bar_colors = [colors.get(family, "#444444") for family in test_results["model_family"]]
    plt.barh(test_results["model"], test_results["f1_ai"], color=bar_colors)
    plt.xlabel("Held-out test F1 for AI class")
    plt.xlim(0, 1.05)
    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.savefig(RESULTS_DIR / "course_vs_transfer_test_f1.png", dpi=200)
    plt.close()

    summary = {
        "dataset_counts": df["label_name"].value_counts().to_dict(),
        "split_counts": df["split"].value_counts().to_dict(),
        "results_file": str(RESULTS_DIR / "course_vs_transfer_comparison.csv"),
        "test_results": test_results.to_dict(orient="records"),
    }
    (RESULTS_DIR / "course_vs_transfer_summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
    print("\nHeld-out test comparison:")
    print(test_results.to_string(index=False))
    print("\nWrote comparison files to", RESULTS_DIR)


if __name__ == "__main__":
    main()
```